Chapter 8 — Individual Assignment

Employee Directory with Vue 3, Axios, Express and MySQL

Cross Platform Application Development · Faculty of Computing, Universiti Teknologi Malaysia

Item	Details
Course	Web Application Development
Chapter	8 — Connecting Vue to Backend
Assessment Type	Individual Assignment (no group submission)

1. Overview

In this assignment you will build a fully functional single-page web application — an Employee Directory — that performs complete CRUD (Create, Read, Update, Delete) operations against a real REST backend that you also build. The front-end must be written in Vue 3 using the Composition API, communicate with the backend using the Axios HTTP client, and persist data in a MySQL database accessed through a Node.js + Express API.

The assignment consolidates the four learning outcomes of Chapter 8 (connecting Vue to a backend, using Axios, handling forms in Vue, and performing CRUD against a real database) into a single realistic deliverable. You are expected to demonstrate not only that your application works, but also that your code is clean, your validation is robust, your SQL is parameterised, and your user experience is considerate.

2. Learning Outcomes

On successful completion of this assignment, you will be able to:

1. Connect a Vue 3 application to an external REST API and manage the request / response lifecycle.
2. Configure and use the Axios HTTP client, including interceptors for logging and centralised error handling.
3. Build Vue forms that capture user input using v-model, validate input with custom rules, and communicate results back to a parent component via events.
4. Build a small Express REST API that performs full CRUD against a MySQL database using prepared statements, including server-side search and sort via query parameters.

3. Scenario

The Human Resources office of a mid-sized Malaysian organisation needs a lightweight internal tool to manage its employee records. Spreadsheets are no longer sustainable: HR staff need a web interface where they can add new hires, update staff details, mark inactive employees, search by name or department, and see the total active headcount at a glance.

Your task is to deliver version 1 of this system — a web application that the HR office can run locally during a pilot phase. The data must persist in a real database (MySQL), accessed through a small Node.js + Express API that you also write. This mirrors the architecture used in production: the browser never talks to the database directly.

4. Functional Requirements

4.1 Data model

Each employee record must contain at least the following fields:

- **empId** — unique employee identifier (e.g. EMP001).
- **name** — full name of the employee.
- **email** — organisational email address.
- **department** — e.g. IT, HR, Finance, Marketing, Operations.
- **position** — job title.
- **hireDate** — ISO date (YYYY-MM-DD).
- **salary** — monthly salary in Ringgit Malaysia (RM).
- **active** — boolean; indicates whether the employee is still employed.

4.2 CRUD operations

Your Express API must support the full CRUD lifecycle against the /employees endpoint:

Operation	HTTP Method / Path	Required Behaviour
Create	POST /employees	Add a new employee after passing client-side validation. Clear the form on success; show the new record in the list immediately. INSERT into MySQL via a prepared statement.
Read (list)	GET /employees	Fetch and render all records on page load (SELECT * FROM employees).
Read (search)	GET /employees?q=…	Server-side search using SQL LIKE on at least three columns (e.g. name, empId, email).

Operation	HTTP Method / Path	Required Behaviour
Read (sort)	GET /employees?sortBy=&order=	Server-side ORDER BY on at least two whitelisted columns (e.g. name, hireDate, salary). Never let the client choose an arbitrary column name.
Update	PUT /employees/:id	Load the selected record into the form, allow editing, then save (UPDATE … WHERE id = ?) and refresh the list.
Delete	DELETE /employees/:id	Ask the user to confirm before deletion; refresh the list after success.

4.3 Form validation (client-side)

The Add / Edit Employee form must validate input before any request is sent. Minimum required rules:

Field	Validation Rule
Employee ID	Required. Must match pattern EMP followed by 3–5 digits (regex ^EMP[0-9]{3,5}$).
Name	Required. At least 3 characters.
Email	Required. Must match a valid email pattern.
Department	Required (selected from a dropdown).
Position	Required.
Hire Date	Required. Cannot be in the future.
Salary (RM)	Required. Numeric. Between 1,500 (national minimum wage) and 50,000.

Invalid fields must display a clear inline error message. The form must not submit while any field is invalid. Use v-model with the .trim and .number modifiers where appropriate.

4.4 Axios integration

- Use a single Axios instance (baseURL, timeout, default headers) as a reusable service module — not inline axios.get(...) calls inside components.
- Implement a request interceptor that logs the method and URL of every outgoing request.
- Implement a response interceptor that maps server or network errors to a single human-readable error message usable by components.
- All HTTP calls must be async / await (not nested .then chains).

4.5 User experience

- The table or card list must clearly show active vs inactive employees (e.g. badge / colour).

- Show a loading state while a request is in flight, and an error banner if a request fails.
- Format salary using Malaysian Ringgit (RM) — for example via Intl.NumberFormat("ms-MY", { style: "currency", currency: "MYR" }).
- The layout must be responsive — usable on both a laptop (≥1024px) and a tablet (≥768px) viewport.

5. Technical Requirements

- Front-end: Vue 3 with the Composition API (<script setup> syntax), built with Vite.
- HTTP client: Axios (with interceptors, as described in §4.4).
- Back-end: Node.js + Express, in a server/ folder inside your project.
- Database driver: the mysql2 package, using its Promise-based pool API. ALL queries must use prepared statements (? placeholders) — never string concatenation.
- Database: MySQL (Laragon). Provide a sql/schema.sql script that creates the database, table, and seed data; the grader will run this script before running your app.
- Component structure: at minimum, separate components for (a) the Add / Edit form and (b) the employee list. A third component for search / sort controls is strongly encouraged.
- State management: parent component holds the employees array; children communicate via props and emits. Vuex / Pinia is not required.
- Seed data: sql/schema.sql must insert at least seven (7) sample employees across at least three (3) different departments, with at least one marked as inactive.
- Your project must run after npm install, executing sql/schema.sql once, and starting both servers (Express on :3001, Vite on :5174).

6. Deliverables

Submit one (1) ZIP archive named:

`Chapter8_<MatricNumber>_<FullName>.zip`

The archive must contain:

5. The full project source code (src/, server/, sql/, index.html, package.json, vite.config.js).
6. A README.md in the project root with: your name, matric number, setup instructions, and any notes on decisions you made or features you extended.
7. A short report (report.pdf, maximum 3 pages) covering: (a) screenshots of your working UI, (b) a brief explanation of how your code satisfies each of the four Chapter 8 learning outcomes, and (c) any challenges you faced and how you resolved them.

Do NOT include the node_modules/ folder or the dist/ build output. Both are recreated by npm install.

7. Grading Summary

Your submission will be graded using the detailed rubric supplied separately (grading_rubric.xlsx). The high-level weighting is:

Criterion	Weight	Mapping
Database + Express API (schema.sql runs, endpoints reachable, prepared statements used)	10%	LO 1
Axios service module (single instance, interceptors, async/await)	15%	LO 2
Form handling and validation (Vue form, rules, inline errors)	20%	LO 3
Full CRUD persisted in MySQL with server-side search and sort	25%	LO 4
Code quality (structure, naming, reusability, comments)	10%	All
User experience (responsive layout, loading / error states, RM formatting)	10%	All
Report and README	10%	All

8. Academic Integrity

This is an individual assignment. You may discuss high-level concepts with classmates but all code you submit must be written by you. Copying code from another student, or submitting code that you did not substantially understand, constitutes plagiarism and will be dealt with under the UTM Academic Integrity Policy.

You may consult official documentation (Vue, Vite, Axios, Express, mysql2, MySQL), tutorials, and Stack Overflow answers, but any such sources must be credited in your report. Use of AI assistants is permitted only for explanations and debugging — the final code architecture and logic must be your own, and you must be able to explain every line of your submission during the viva.

— End of Assignment Brief —